



การกำหนดและจัดการ EVENT

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

อีเวนต์ของ JavaScript

อีเวนต์ (Event) คือเหตุการณ์หรือการกระทำบางอย่างที่เกิดขึ้น เช่น การคลิกหรือเคลื่อนย้ายเมาส์, การกดปุ่มคีย์บอร์ด, การเลือกหรือไม่เลือกรายการ เป็นต้น โดยทั่วไปเรามักใช้อีเวนต์ที่เกิดขึ้นเป็นตัวกำหนดการตอบสนองหรือการกระทำบางอย่าง สำหรับรายชื่ออีเวนต์ที่น่าสนใจซึ่งเราอาจได้นำไปใช้งานอยู่บ่อยๆ มีดังนี้

อีเวนต์	เกิดอีเวนต์เมื่อ	อิลิเมนต์ที่สามารถเกิดอีเวนต์
click	คลิกที่อิลิเมนต์	เกิดกับอิลิเมนต์ส่วนใหญ่ที่มีรูปร่าง
blur	ย้ายโฟกัสไปยังอิลิเมนต์อื่น	button, input, select, textarea, body
change	เมื่อเปลี่ยนแปลงการเลือกรายการ	select, checkbox, radio
error	มีข้อผิดพลาดในการโหลดรูปภาพ	img
focus	อิลิเมนต์ได้รับโฟกัส	button, input, select, textarea, body
input	ใส่ข้อมูลลงไปหรือลบออก	input, textarea
keydown	ปุ่มคีย์บอร์ดถูกกด	อิลิเมนต์ของฟอร์มและ body
keypress	ปุ่มคีย์บอร์ดถูกกด (เกิดหลัง keydown)	อิลิเมนต์ของฟอร์มและ body

อีเวนต์ของ JavaScript

อีเวนต์	เกิดอีเวนต์เมื่อ	อิลิเมนต์ที่สามารถเกิดอีเวนต์
keyup	ปล่อยมือจากคีย์บอร์ด (เกิดหลัง keypress)	อิลิเมนต์ของฟอร์มและ body
load	เมื่อเอกสารหรือภาพถูกโหลดอย่างสมบูรณ์	body, img
mousedown	กดปุ่มของเมาส์	เกิดกับอิลิเมนต์ส่วนใหญ่
mousemove	เคลื่อนย้ายเมาส์บนอิลิเมนต์นั้น	เกิดกับอิลิเมนต์ส่วนใหญ่
mouseout	ย้ายเมาส์ออกจากอิลิเมนต์นั้น	เกิดกับอิลิเมนต์ส่วนใหญ่
mouseover	นำเมาส์มาวางเหนืออิลิเมนต์นั้น	เกิดกับอิลิเมนต์ส่วนใหญ่
mouseup	ปล่อยปุ่มเมาส์ที่กด (เกิดหลัง mousedown)	เกิดกับอิลิเมนต์ส่วนใหญ่
reset	เมื่อเคลียร์ข้อมูลในฟอร์ม	form
select	เลือกข้อความในอิลิเมนต์	input, textarea
submit	ส่งข้อมูลจากฟอร์มไปยังเซิร์ฟเวอร์	form
unload	ยกเลิกการโหลดเอกสาร	body

อีเวนต์ของ JavaScript

สำหรับใน JavaScript เราสามารถกำหนดอีเวนต์ให้กับอิลิเมนต์ได้หลายวิธี แต่ในที่นี้จะเน้นเฉพาะแบบแอตทริบิวต์ เพราะเป็นวิธีหลักที่จะนำไปใช้ร่วมกับ JSX โดยแนวทางการกำหนดอีเวนต์ใน JavaScript มีดังนี้

- เราต้องสร้างฟังก์ชันสำหรับเป็นตัวจัดการอีเวนต์ ซึ่งเราอาจเรียกฟังก์ชันแบบนี้ว่า Event Handler ทั้งนี้ เราอาจสร้างเป็น Regular Function หรือแบบ Arrow Function ก็ได้ เช่น

```
function buttonClick() {           //Regular Function
    alert('You clicked button')
}
const imgError = () => {           //Arrow Function
    alert('Image Error')
}
```

- ที่แท็กของอิลิเมนต์ซึ่งต้องการตอบสนองอีเวนต์ เราก็เพิ่มแอตทริบิวต์ onxxx เมื่อ xxx คือชื่ออีเวนต์ที่เกิดขึ้น เช่น onclick, onfocus เป็นต้น (ปกติเราจะเขียนด้วยตัวพิมพ์เล็กทั้งหมด) พร้อมระบุค่าเป็นชื่อฟังก์ชันที่ต้องการเรียกขึ้นมาทำงานเมื่อเกิดอีเวนต์ เช่น

```
<button onclick="buttonClick()">OK</button>

```

การกำหนดอีเวนต์สำหรับ React JSX

สำหรับ React JSX จะมีวิธีการกำหนดอีเวนต์ที่แตกต่างไปจากรูปแบบปกติของ JavaScript ในบางกรณี ซึ่งสามารถสรุปหลักการที่สำคัญในเบื้องต้นได้ดังนี้

- หากเป็นรูปแบบของ HTML เราจะเขียนแอตทริบิวต์ที่ใช้ระบุชื่อ Event Handler เป็นตัวพิมพ์เล็กทั้งหมด เช่น onclick แต่ใน React JSX ให้เขียนแบบ camelCase เช่นการเปรียบเทียบแอตทริบิวต์บางส่วนดังนี้

แอตทริบิวต์แบบ HTML	แอตทริบิวต์แบบ JSX
onclick	onClick
onblur	onBlur
onfocus	onFocus
onchange	onChange
onselect	onSelect
onmousedown	onMouseDown
...	...

การกำหนดอีเวนต์สำหรับ React JSX

- หากเราสร้าง Event Handler ภายใน Class Component มันจะกลายเป็นเมธอด ดังนั้น ต้องอ้างอิง Event Handler ผ่านคำว่า this เหมือนกับการอ้างอิงเมธอดภายในคลาสตามปกติ ในลักษณะดังนี้

```
Class Button extends Component {  
  onClickButtonOK() {      //Event Handler  
    alert('OK')  
  }  
  
  onClickButtonCancel = () => {      //Event Handler แบบ Arrow Function  
    alert('Cancel')  
  }  
}
```

การกำหนดอีเวนต์สำหรับ React JSX

```
render() {  
  return (  
    <div>  
      <button onClick={this.onClickButtonOK}>OK</button><br/>  
      <button onClick={this.onClickButtonCancel}>Cancel</button>  
    </div>  
  )  
}
```

- กรณีของ Function Component ให้ระบุแค่ชื่อ Event Handler โดยไม่ต้องมีคำว่า this

```
function Button() {  
  onClickButtonCancel = () => {    //Event Handler แบบ Arrow Function  
    ...  
  }  
  return <button onClick={onClickButtonOK}>OK</button>  
}
```

การกำหนดอีเวนต์สำหรับ React JSX

ตัวอย่างการใช้งานจะเป็นการกำหนดอีเวนต์ให้กับคอมโพเนนต์ Banner ซึ่งใช้แสดงป้ายโฆษณาแบบสุ่ม โดยเมื่อคลิกที่ป้ายดังกล่าว ก็จะเปิดไปยังเว็บไซต์ที่กำหนด

1. สร้างโฟลเดอร์ banners เพิ่มลงในโฟลเดอร์ public แล้วลองหาภาพป้ายโฆษณาอย่างน้อย 3 ภาพ ใส่เก็บไว้ในโฟลเดอร์ดังกล่าว
2. เพิ่มไฟล์ใหม่เข้ามาในโฟลเดอร์ src ซึ่งในที่นี้กำหนดชื่อเป็น banner.js เพื่อสร้างคอมโพเนนต์ รวมถึงขั้นตอนจัดการอีเวนต์ก็จะกำหนดไว้ในไฟล์นี้ทั้งหมด โดยเป็นการสร้าง Function Component สำหรับแสดงป้ายโฆษณา การกำหนดอีเวนต์เมื่อคลิกที่ป้าย เป็นต้น โดยกำหนดโค้ดลงในไฟล์ banner.js ดังนี้

react/app1/src/banner.js

```
import React from 'react'
function randInt(min, max) {
  let x = (max - min) + 1
  return min + Math.floor(Math.random() * x)
}
export default function Banner() {
  const r = randInt(1, 3)
```


การกำหนดอีเวนต์สำหรับ React JSX

```
let banner = ""
let url = ""

switch (r) {
  case 1:
    banner = './banners/react.jpg'
    url = 'https://www.reactjs.org'
    break
  case 2:
    banner = './banners/react-native.jpg'
    url = 'https://www.reactnative.com'
    break
  case 3:
    banner = './banners/node.jpg'
    url = 'https://www.nodejs.org'
    break
  default: break
}
```

```
const bannerClick = () => { //Event Handler (Arrow Func)
  window.open(url)
  //window.location.href = url
}

return (
  <div style={{margin: '20px', textAlign: 'center'}}>
    <img src={banner} onClick={bannerClick} style={{cursor:'pointer'}}
      alt="banner"/>
  </div>
)
```

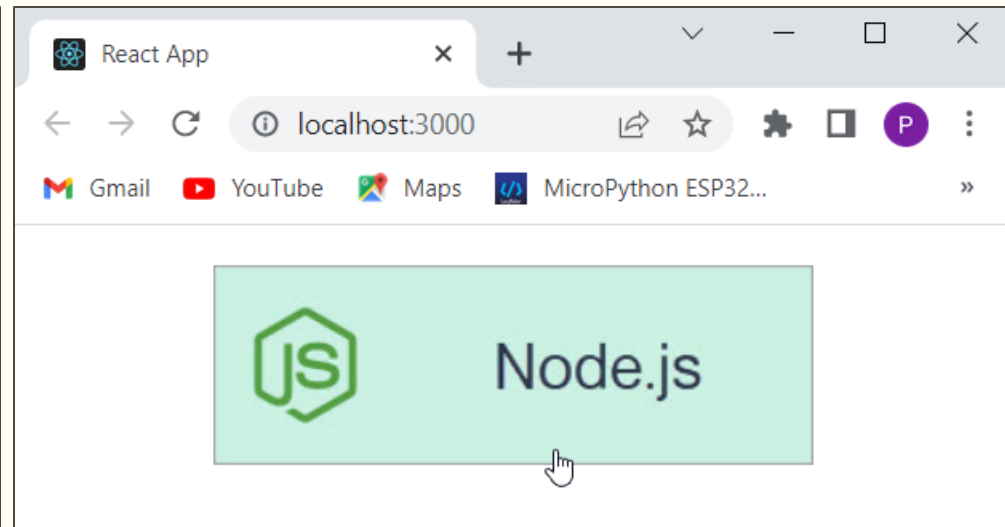
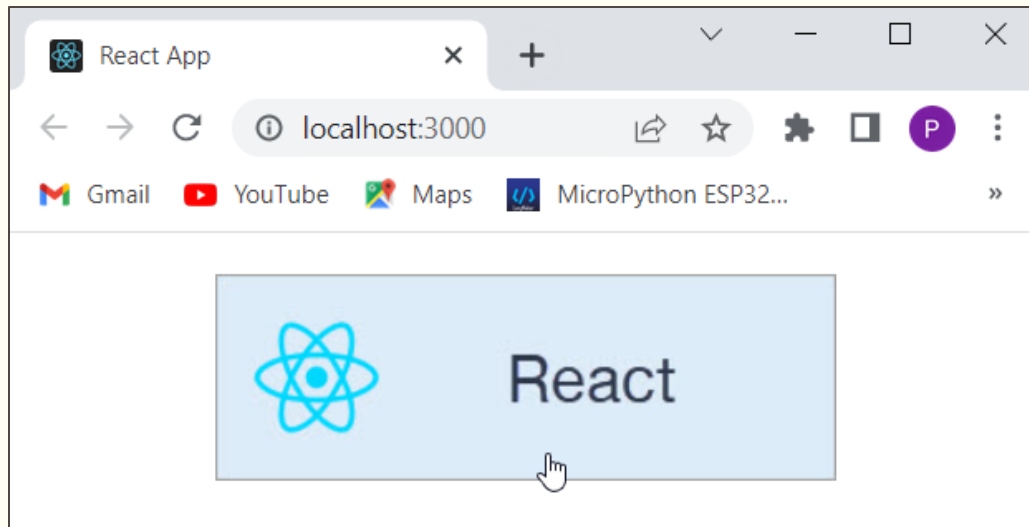
การกำหนดอีเวนต์สำหรับ React JSX

3. ที่ไฟล์ App.js เราก็แค่แสดงคอมโพเนนต์ Banner ตามปกติ

```
react/app1/src/App.js
```

```
import React from 'react'
import Banner from './banner'

export default function App() {
  return <Banner/>
}
```



ปัญหาการใช้ `this` ใน Class Component

กรณีที่เรารสร้าง Class Component แล้วสร้างตัวจัดการอีเวนต์แบบ Regular Function จะมีปัญหาบางอย่างคือไม่สามารถใช้ `this` เพื่ออ้างอิงเมธอดหรือพร็อพเพอร์ตี้อื่นๆ ภายในคลาสนั้นได้ เช่น ลองดูโค้ดต่อไปนี้ ถ้าแสดงผลแล้วคลิกปุ่ม OK จะเกิดข้อผิดพลาด

ตัวอย่างการใช้งาน

react/app1/src/class-components.js

```
import React, {Component} from 'react'
export default class Button extends Component {
  showAlert(msg) {
    alert(msg)
  }

  onClickButtonOK() {
    //Event Handler แบบ Regular Function
    this.showAlert('Hello')
  }
}
```

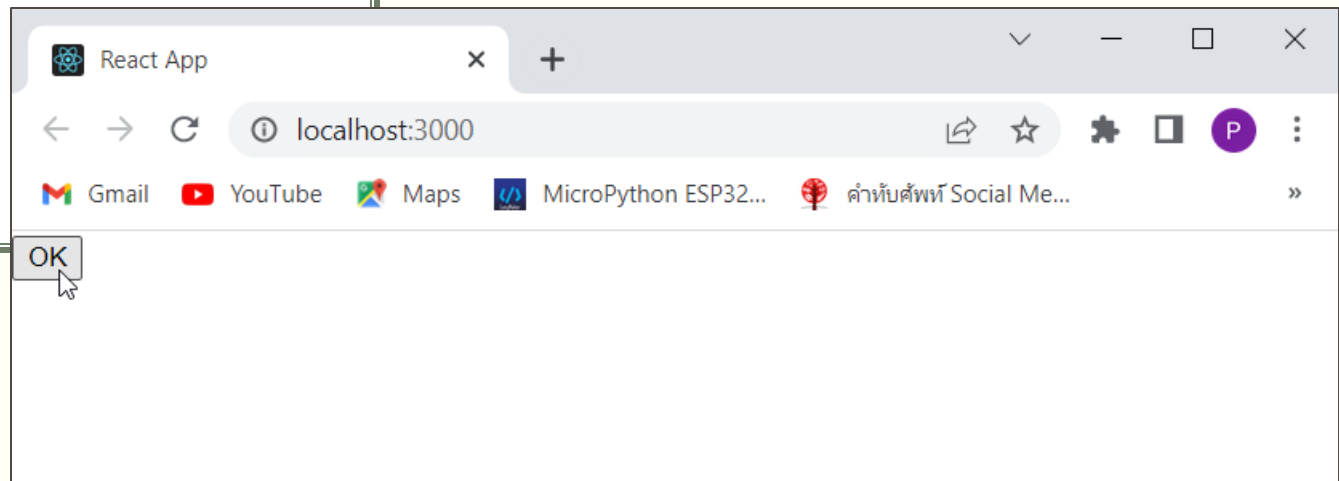
```
render() {
  return <button onClick={this.onClickButtonOK}>OK</button>
}
}
```

ปัญหาการใช้ **this** ใน Class Component

react/app1/src/App.js

```
import React from 'react'
import Button from './class-components'

function App() {
  return <Button/>
}
export default App
```



ปัญหาการใช้ `this` ใน Class Component

ปัญหาดังกล่าว เกิดจากการที่ `onClickButtonOK()` ซึ่งเป็น Event Handler ในแบบ Regular Function ไม่สามารถใช้ `this` เพื่ออ้างถึงเมธอด `showAlert()` ได้ ทั้งนี้หากเราต้องการแก้ปัญหาก็อาจเพิ่มคอนสตรักเตอร์เข้ามาในคลาส จากนั้นเรียกเมธอดที่เป็น Event Handler แล้วตามด้วย `bind(this)` เช่น จากโค้ดที่ผ่านมา อาจแก้ไขเป็นดังนี้

ตัวอย่างการใช้งาน

react/app1/src/class-components.js

```
import React, {Component} from 'react'
export default class Button extends Component {
  constructor() {
    super()
    this.onClickButtonOK = this.onClickButtonOK.bind(this)
  }

  showAlert(msg) {
    alert(msg)
  }
}
```

```
onClickButtonOK() {
  this.showAlert('Hello') //ต่อไปก็ใช้ this ได้แล้ว
}

render() {
  return <button onClick={this.onClickButtonOK}>OK</button>
}
}
```

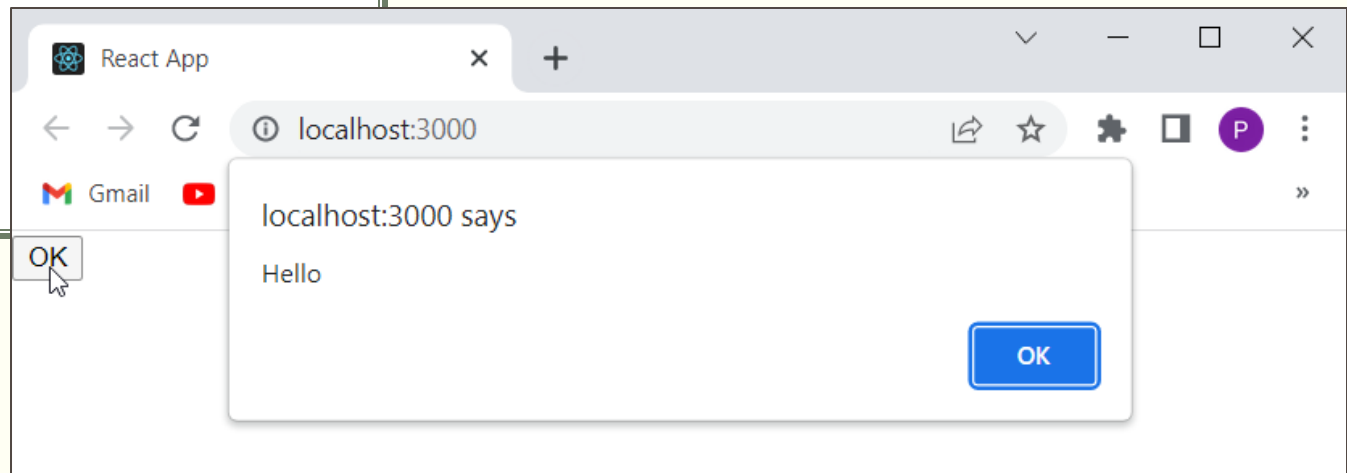
ปัญหาการใช้ **this** ใน Class Component

react/app1/src/App.js

```
import React from 'react'
import Button from './class-components'

function App() {
  return <Button/>
}

export default App
```



ปัญหาการใช้ `this` ใน Class Component

เมื่อทดสอบโค้ดข้างบนก็จะใช้งานได้ตามปกติ อย่างไรก็ตาม ปัญหาดังที่กล่าวมาจะเกิดเฉพาะกรณีที่เรากำหนด Event Handler ในแบบ Regular Function เท่านั้น แต่ไม่เกิดกับกรณีที่กำหนด Event Handler ในแบบ Arrow Function เช่น โค้ดต่อไปนี้จะไม่เกิดข้อผิดพลาด

ตัวอย่างการใช้งาน

react/app1/src/class-components.js

```
import React, {Component} from 'react'
export default class Button extends Component {
  showAlert(msg) {
    alert(msg)
  }

  onClickButtonOK = () => { //Event Handler แบบ Arrow Function
    this.showAlert('Hello') //ใช้ this ได้เลย โดยไม่เกิดปัญหา
  }
```

```
render() {
  return <button onClick={this.onClickButtonOK}>
    OK</button>
  }
}
```

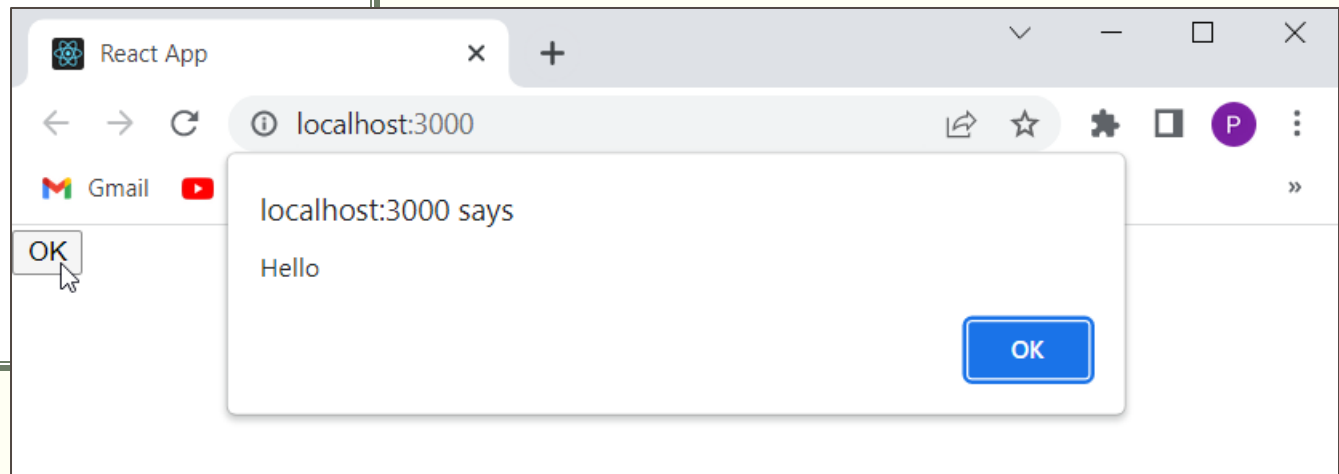
ปัญหาการใช้ `this` ใน Class Component

react/app1/src/App.js

```
import React from 'react'
import Button from './class-components'

function App() {
  return <Button/>
}

export default App
```



หากเราไม่ต้องการเสียเวลาสร้างคอนสตรัคเตอร์ให้กับคลาสเพื่อแก้ปัญหาดังที่กล่าวมา ก็ควรกำหนด Event Handler แบบ Arrow Function ก็จะช่วยลดความยุ่งยากลงไปได้ ส่วนกรณีของ Function Component เราไม่ต้องใช้ `this` จึงไม่เกิดปัญหา แม้จะกำหนด Event Handler ในแบบ Regular Function ก็ตาม

การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

เราสามารถส่งข้อมูลบางอย่างจากอีลิเมนต์ที่เกิดอีเวนต์ ไปยังฟังก์ชันตัวจัดการอีเวนต์ เพื่อนำไปใช้งานตามต้องการ ทำให้ฟังก์ชันมีความยืดหยุ่นมากขึ้น ซึ่งมีแนวทางดังนี้

- ที่ฟังก์ชันจัดการอีเวนต์ ให้เรากำหนดพารามิเตอร์เพื่อรับข้อมูล และภายในฟังก์ชันก็นำพารามิเตอร์ไปใช้งานตามวิธีการปกติ เช่น

```
class Button extends Component {  
    function add(num1, num2) { //Event Handler  
        ...  
    }  
    ...  
}
```

การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

- ที่อีลิเมนต์ซึ่งเกิดอีเวนต์ เราสามารถผ่านค่าไปยังฟังก์ชันจัดการอีเวนต์ได้ 2 วิธี คือ
 - วิธีที่ 1 กำหนดค่าในแบบ Arrow Function เช่น

```
<button onClick={() => this.add(10, 20)}>Add</button>
```

/* จากโค้ด

() => this.add(10, 20)

คือ Arrow Function ที่ใช้ระบุ Event Handler (add) และ (10, 20) คือ อาร์กิวเมนต์

*/

- วิธีที่ 2 โดยการใช้เมธอด bind() ดังโค้ดถัดไป โดยต้องเรียกเมธอด bind() ผ่านฟังก์ชันเป้าหมายที่เป็นตัวจัดการอีเวนต์ และต้องระบุ this เป็นอาร์กิวเมนต์ตัวแรกเสมอ

```
<button onClick={this.add.bind(this, 10, 20)}>Add</button>
```

การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

สำหรับแนวทางการทดสอบเพื่อผ่านค่าอาร์กิวเมนต์ไปยังตัวจัดการอีเวนต์ มีดังนี้

1. เพิ่มไฟล์ calculator.js เข้ามาในโฟลเดอร์ src ซึ่งภายในไฟล์นี้จะเป็นการสร้างคอมโพเนนต์เพื่อกำหนดอีเวนต์เมื่อคลิกปุ่มเครื่องหมาย +, -, *, / จะส่งผ่านค่าอาร์กิวเมนต์ที่เป็นเครื่องหมายบนปุ่มนั้นไปยัง Event Handler เพื่อคำนวณ

react/app1/src/calculator.js

```
import React from 'react'

export class Calculator extends React.Component {
  n1 = 15
  n2 = 5

  calculate = (op) => {
    let c = `${this.n1} ${op} ${this.n2}`
    let r = eval(c)
    alert(`${c} = ${r}`)
  }
}
```

```
render() {
  return (
    <div style={{textAlign:'center', marginTop:20}}>
      <span>{this.n1}</span>&nbsp;
      <button onClick={() => this.calculate('+')}>+</button>&nbsp;
      <button onClick={() => this.calculate('-')}>-</button>&nbsp;
      <button onClick={this.calculate.bind(this, '*')}>*</button>&nbsp;
      <button onClick={this.calculate.bind(this, '/')}>/</button>&nbsp;
      <span>{this.n2}</span>
    </div>
  )
}
```

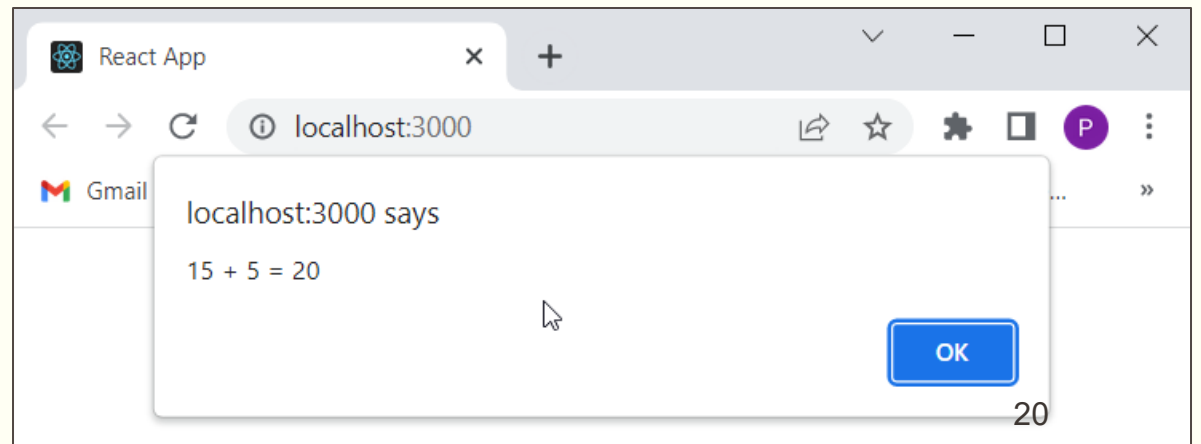
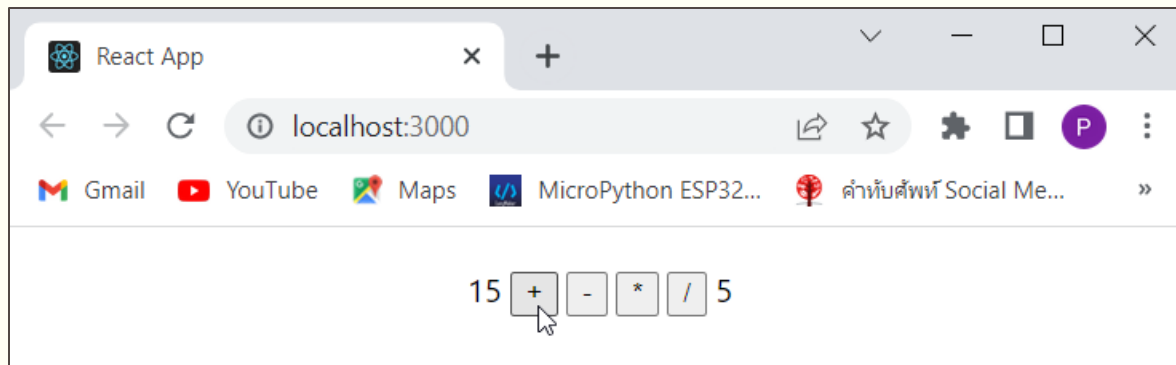
การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

2. ที่ไฟล์ App.js เราก็แค่แสดงคอมโพเนนต์ตามปกติ

react/app1/src/App.js

```
import React from 'react'
import {Calculator} from './calculator'

export default function App() {
  return <Calculator/>
}
```



การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

หากเราสร้างคอมโพเนนต์ในแบบฟังก์ชัน ก็มีแนวทางดังนี้

react/app1/src/calculator.js

```
import React from 'react'

export function .....() {
  const n1 = 20
  const n2 = 4

  const calculate = (op) => {
    let c = `${n1} ${.....} ${n2}`
    let r = eval(c)
    alert(`${.....} = ${.....}`)
  }
}
```

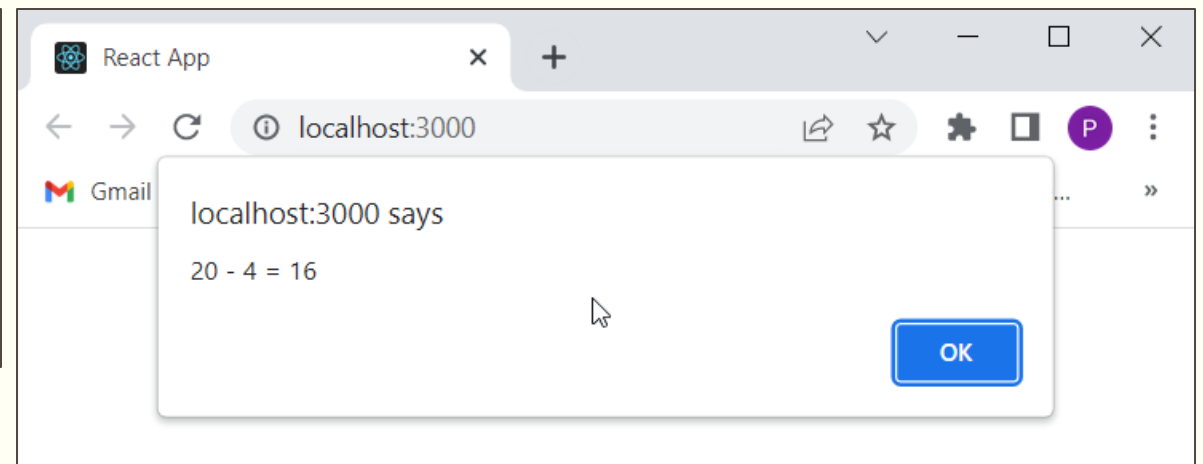
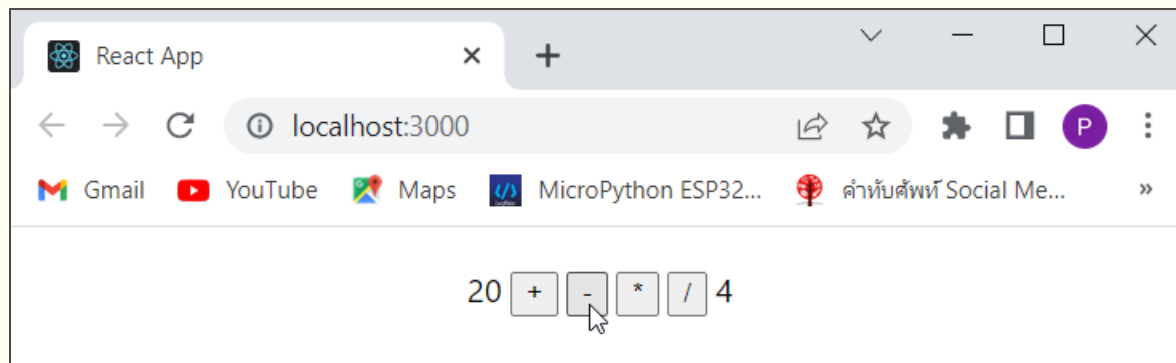
```
return (
  <div style={{textAlign:'center', marginTop:20}}>
    <span>{.....}</span>&nbsp;
    <button onClick={() => .....}>+</button>&nbsp;
    <button onClick={() => .....}>-</button>&nbsp;
    <button onClick={() => .....}>*</button>&nbsp;
    <button onClick={() => .....}>/</button>&nbsp;
    <span>{.....}</span>
  </div>
)
}
```

การผ่านค่าอาร์กิวเมนต์ไปยังฟังก์ชันจัดการอีเวนต์

react/app1/src/App.js

```
import React from 'react'
import ..... from './calculator'

export default function App() {
  return <Calculator2/>
}
```



การตรวจสอบข้อมูลของอีเวนต์

ในบางกรณี เราอาจต้องการตรวจสอบข้อมูลบางอย่างเกี่ยวกับอีเวนต์ที่เกิดขึ้น เช่น ชนิดอีเวนต์ หรืออีลีเมนต์ที่เกิดอีเวนต์ หรือปุ่มเมาส์/คีย์บอร์ดที่ถูกกด เป็นต้น ซึ่งในลักษณะนี้ เราต้องผ่านออบเจกต์ของอีเวนต์ไปยังฟังก์ชันที่เป็นตัวจัดการอีเวนต์ แต่มีข้อกำหนดที่แตกต่างจากการผ่านค่าอาร์กิวเมนต์ทั่วไป ดังแนวทางต่อไปนี้

- ที่ฟังก์ชันซึ่งใช้เป็นตัวจัดการอีเวนต์ ให้ระบุพารามิเตอร์สำหรับการรับออบเจกต์ของอีเวนต์เข้ามาในฟังก์ชัน และหากมีพารามิเตอร์ที่ใช้รับข้อมูลอื่นๆ ด้วย ให้วางตำแหน่งพารามิเตอร์สำหรับออบเจกต์ของอีเวนต์ไว้ในลำดับสุดท้าย เช่น

```
function onClickButton1(ev) {  
    ...  
}  
function onClickButton2(a, b, ev) { //ev ใช้รับออบเจกต์ของอีเวนต์  
    ...  
}  
const onClickLink = (url, ev) => { //กรณีที่ใช้ Arrow Function  
    ...  
}
```

การตรวจสอบข้อมูลของอีเวนต์

- ที่อีลีเมนต์ซึ่งเกิดอีเวนต์ เราสามารถผ่านออบเจกต์ไปยังฟังก์ชันที่เป็นตัวจัดการอีเวนต์ได้ 2 วิธี คือ
 - วิธีที่ 1 กำหนดค่าในแบบ Arrow Function (ใช้ ev คือออบเจกต์ของอีเวนต์) แต่หากตัวจัดการอีเวนต์มีพารามิเตอร์ ev เพียงตัวเดียว เราอาจจะบุแค่ชื่อฟังก์ชันก็ได้ (ไม่จำเป็นต้องกำหนดพารามิเตอร์ในแบบ Arrow Function) เช่น

```
//กรณีสร้างเป็น Class Component
<button onClick={this.onClickButton1}>Click</button>
<button onClick={(ev) => this.onClickButton1(ev)}>Click</button>

<button onClick={(ev) => this.onClickButton2(10, 20, ev)}>Click</button>

<a href=" " onClick={(ev) => this.linkClick('http://ex.com', ev)}>
  Go To...
</a>
```


การตรวจสอบข้อมูลของอีเวนต์

//กรณีสร้างเป็น Function Component (ไม่ต้องใช้ this)

```
<button onClick={onClickButton1}>Click</button>
```

```
<button onClick={(ev) => onClickButton1(ev)}>Click</button>
```

```
<button onClick={(ev) => onClickButton2(10, 20, ev)}>Click</button>
```

```
<a href=" " onClick={(ev) => linkClick('http://ex.com', ev)}>
```

```
  Go To...
```

```
</a>
```

การตรวจสอบข้อมูลของอีเวนต์

- วิธีที่ 2 หากเป็น Class Component ก็ใช้เมธอด bind() แต่ **ไม่** ต้องระบุออบเจกต์ของอีเวนต์เพราะในกรณีนี้ React จะถือพารามิเตอร์ตัวสุดท้ายที่เรากำหนดไว้ที่ฟังก์ชันคือออบเจกต์ของอีเวนต์โดยอัตโนมัติ (กรณีนี้คือ ev) แต่สำหรับ Function Component ซึ่งใช้เมธอด bind() และ this ไม่ได้ ดังนั้น จะต้องใช้รูปแบบ Arrow Function ตามวิธีที่ 1 เท่านั้น

//สำหรับ Class Component เท่านั้น

```
<button onClick={this.onClickButton1.bind(this)}>Click</button>
```

```
<button onClick={this.onClickButton2.bind(this, 10, 20)}>Click</button>
```

```
<a href=" " onClick={this.linkClick.bind(this, 'http://ex.com')}>
```

```
  Go To...
```

```
</a>
```

การตรวจสอบข้อมูลของอีเวนต์

ออบเจกต์ของอีเวนต์ที่ผ่านเข้าไปในฟังก์ชัน (ในที่นี้แทนด้วย ev) สามารถใช้ในการอ้างอิงข้อมูลต่างๆ เกี่ยวกับอีเวนต์ที่เกิดขึ้นผ่านทางพรีออปเพอร์เตอร์และเมธอดของออบเจกต์ดังกล่าว โดยอันที่น่าสนใจซึ่งเราอาจได้นำไปใช้งานบ่อยๆ คือ

target	อิลิเมนต์เป้าหมายที่เกิดอีเวนต์
type	ชนิดอีเวนต์ที่เกิดขึ้น
button	ตรวจสอบปุ่มของเมาส์ที่ถูกกด โดย 0=ซ้าย, 1=กลาง, 2=ขวา
keyCode หรือ which	รหัสของปุ่มคีย์บอร์ดที่ถูกกด
shiftKey, altKey, ctrlKey	กดปุ่ม Shift, Alt หรือ Control ค้างไว้หรือไม่ ตามลำดับ
preventDefault()	ยกเลิกพฤติกรรมแบบปกติเมื่อเกิดอีเวนต์กับอิลิเมนต์นั้น
stopPropagation()	ป้องกันการเกิด Event Bubbling

การตรวจสอบข้อมูลของอีเวนต์

ตัวอย่างการใช้งาน

react/app1/src/event-data.js

```
import React from 'react'

export class EventData1 extends React.Component {

  onClickStartStop = (ev) => {
    let t = ev.target.innerText
    let s = (t === 'Start') ? 'Stop' : 'Start'
    ev.target.innerText = s
  }

  onClickAdd = (ev) => {
    let t = ev.target.innerText
    let r = eval(t)
    alert(`${t} = ${r}`)
  }
```

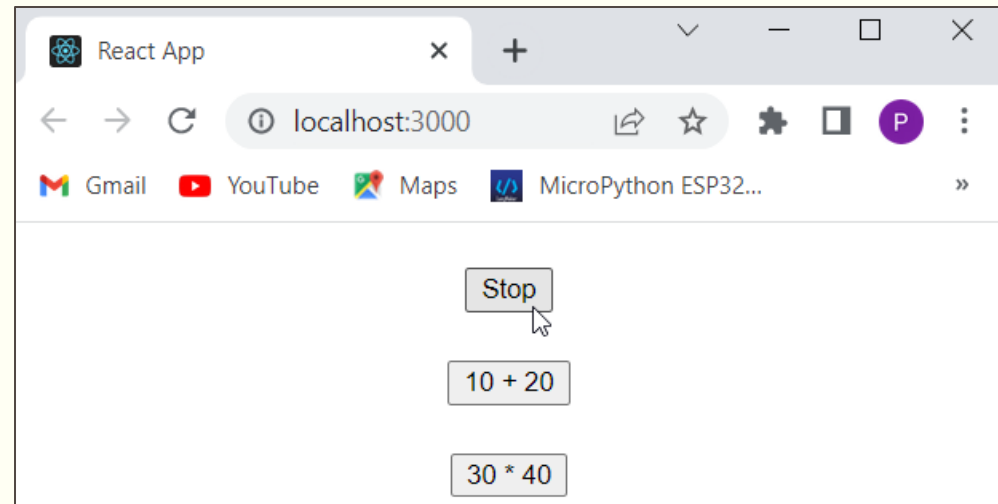
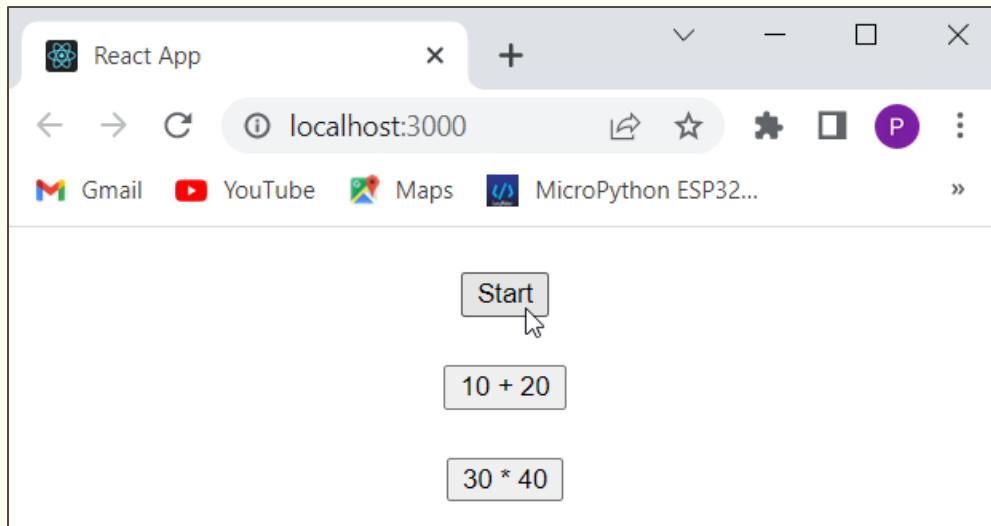
```
render() {
  return (
    <div style={{textAlign:'center', marginTop:20}}>
      <button onClick={(ev) => this.onClickStartStop(ev)}>Start</button>
      <br/><br/>
      <button onClick={(ev) => this.onClickAdd(ev)}>10 + 20</button>
      <br/><br/>
      <button onClick={this.onClickAdd.bind(this)}>30 * 40</button>
      <br/><br/>
    </div>
  )
}
```

การตรวจสอบข้อมูลของอีเวนต์

react/app1/src/App.js

```
import React from 'react'
import {EventData1} from './event-data'

export default function App() {
  return <EventData1/>
}
```



การตรวจสอบข้อมูลของอีเวนต์

หากเราสร้างคอมโพเนนต์ในแบบฟังก์ชัน ก็จะมีแนวทางดังนี้

ตัวอย่างการใช้งาน

react/app1/src/event-data.js

```
import React from 'react'

export function ..... {
  const ..... = (.....) => {
    let t = ev.target.innerText
    let s = (t === 'Start') ? 'Stop' : 'Start'
    ev.target.innerText = .....
  }
}
```

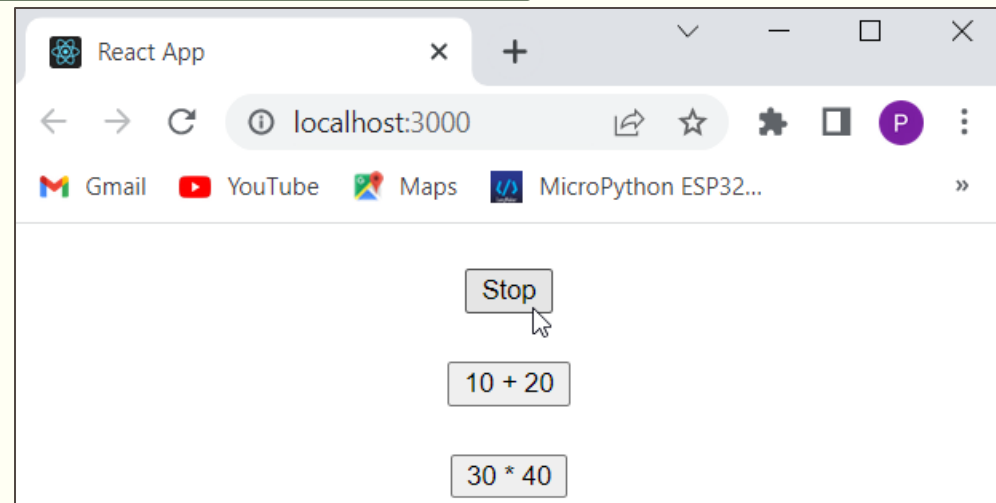
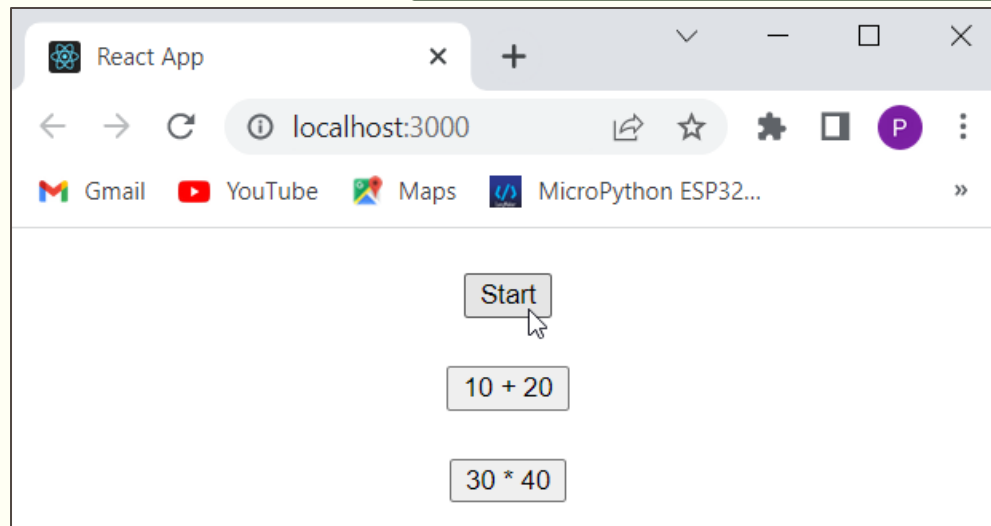
```
const onClickAdd = (ev) => {
  let t = .....
  let r = eval(t)
  alert(`${t} = ${r}`)
}

return (
  <div style={{textAlign:'center', marginTop:20}}>
    <button onClick={(ev) => onClickStartStop(ev)}>.....</button><br/><br/>
    <button onClick={.....}>.....</button><br/><br/>
    <button onClick={.....}>30 * 40</button><br/><br/>
  </div>
)
}
```

การตรวจสอบข้อมูลของอีเวนต์

react/app1/src/App.js

```
import React from .....  
import {EventData2} from .....  
  
export default function App() {  
  return .....  
}
```



การตรวจสอบข้อมูลของอีเวนต์

เรามาดูอีกตัวอย่างเพิ่มเติม ซึ่งเป็นการทำไฮไลต์พื้นหลังของตารางเมื่อเลื่อนเมาส์มาวางเหนือแถวนั้น แล้วถ้านำเมาส์ออกไปก็จะเปลี่ยนกลับไปเป็นสีเดิม

ตัวอย่างการใช้งาน

react/app1/src/table.js

```
import React from 'react'
```

```
export function Table() {
```

```
  const data = [  
    ['JavaScript', 100],  
    ['React', 150],  
    ['React Native', 180],  
    ['Node.js', 200],  
    ['VS Code', 120]
```

```
]
```

```
const onMouseOverRow = (ev) => {  
  ev.target.parentNode.style.backgroundColor = 'yellow'  
}
```

```
const onMouseOutRow = (ev) => {  
  ev.target.parentNode.style.backgroundColor = 'white'  
}
```


การตรวจสอบข้อมูลของอีเวนต์

react/app1/src/table.js (ต่อ)

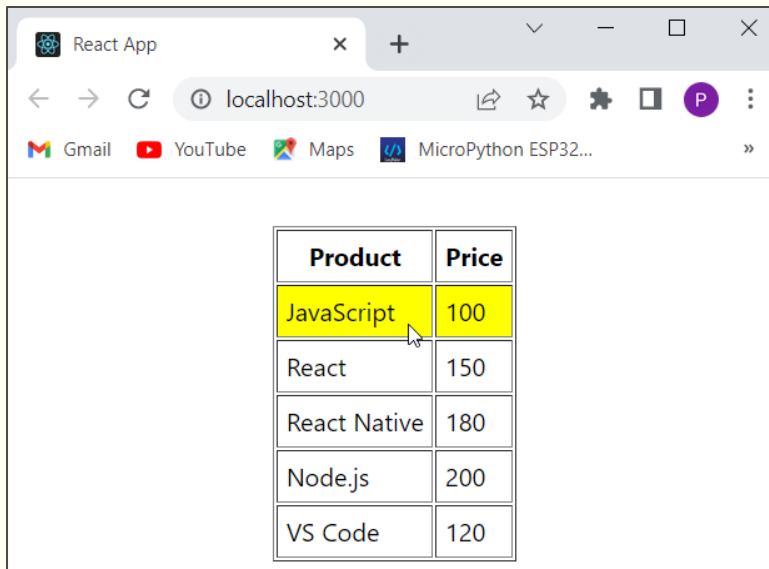
```
return (  
  <table border="1" cellPadding={5} style={{margin: 'auto', marginTop: 30}} >  
    <tr><th>Product</th><th>Price</th></tr>  
    {  
      data.map(item => {  
        return (  
          <tr onMouseOver={onMouseOverRow} onMouseOut={onMouseOutRow}>  
            <td>{item[0]}</td><td>{item[1]}</td>  
          </tr>  
        )  
      })  
    }  
  </table>  
)  
}
```

การตรวจสอบข้อมูลของอีเวนต์

react/app1/src/App.js

```
import React from 'react'
import {Table} from './table'

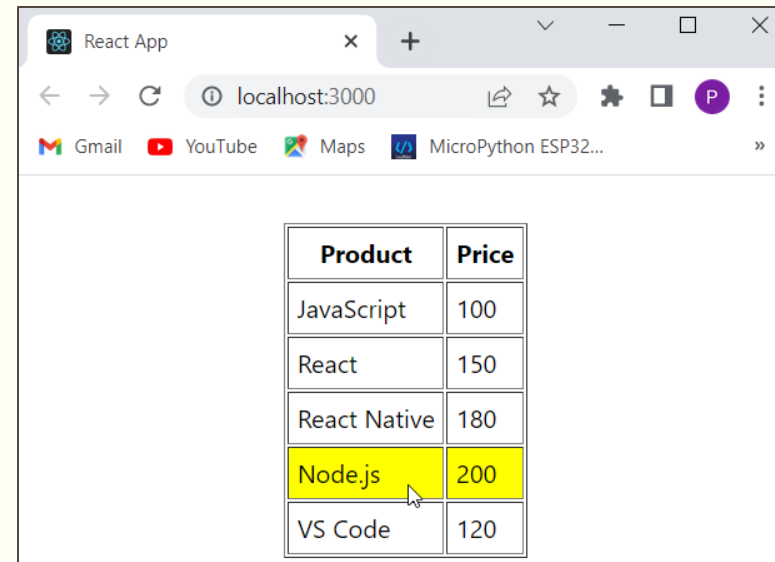
export default function App() {
  return <Table/>
}
```



React App

localhost:3000

Product	Price
JavaScript	100
React	150
React Native	180
Node.js	200
VS Code	120



React App

localhost:3000

Product	Price
JavaScript	100
React	150
React Native	180
Node.js	200
VS Code	120